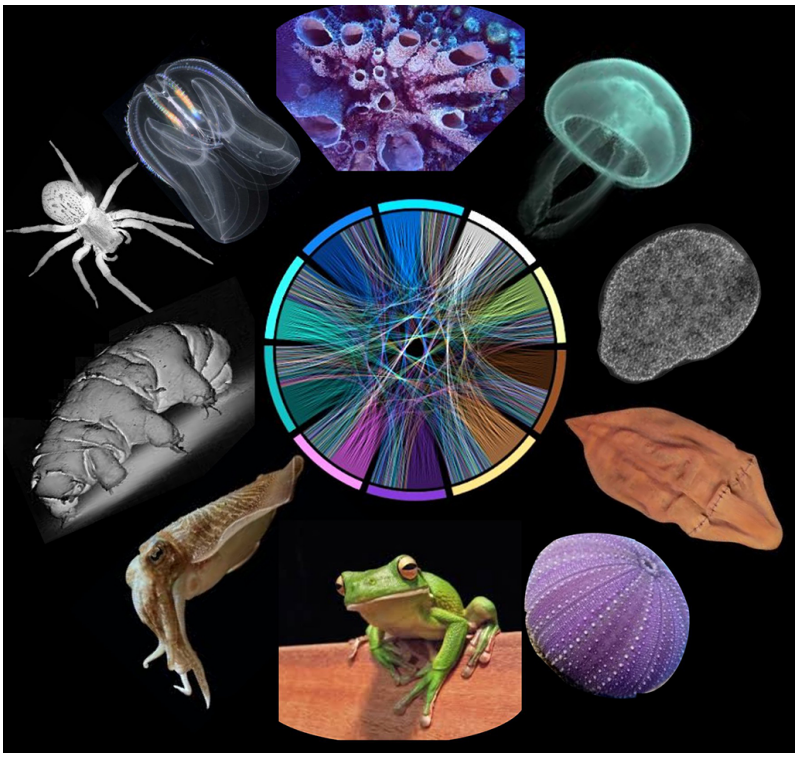
Book of Specifications

GenomeXplore

Platform for exploring and selecting reference genomes and associated raw sequencing data



Project delivery team

Yasmine ALIOUAT Colin PALANDRE Margaux PILLON

Project owners

Julien FOURET Carine REY

Bioinformatics and Biostatistics team (BIBS)

Centre International de Recherche en Infectiologie (CIRI)

M2 Bioinformatics – Lyon 1 University

October 2026

Contents

1 Project presentation **1**

1.1 Context . . . 1

1.2 Objectives . . . 1

1.3 Description of the existing . . . 1

1.4 Validation procedure and acceptance criteria . . . 2

2 Project requirements **2**

2.1 Functional needs . . . 2

2.2 Non-functional needs . . . 4

3 Solution architecture **4**

3.1 Overall architecture . . . 4

3.2 Backend architecture . . . 4

3.3 Frontend architecture . . . 6

3.4 User interface design . . . 7

4 Constraints **8**

4.1 Budget and resources . . . 8

4.2 Schedule . . . 8

4.3 Other constraints . . . 8

5 Project development process **8**

5.1 Planning . . . 8

5.2 Quality control and testing procedures . . . 9

5.3 Documentation . . . 9

6 Responsibilities **9**

6.1 Project owner . . . 9

6.2 Project delivery team . . . 10

A NCBI database description **10**

B Ensembl database description **15**

C NGDC/GWH database description **20**

List of Figures

1 Container diagram of GenomeXplore (local deployment). 5

2 Component diagram of the backend API. 6

3 Component diagram of the frontend. 7

4 Web page design (mock-up): taxonomic search (page 1), assemblies exploration and selection (page 2), raw data exploration and selection (page 3). 7

5 Gantt chart of the project (specifications, tool development, finalization and defense). 8

1 Project presentation

1.1 Context

In order to study the similarities and differences between species, we study their genomes by comparing their sequences, genes, and structure, which can reveal how genes evolved, how species are related to one another and how new functions appeared. This discipline is called *comparative genomics*. Such a comparison can be accurate only if the genomes are assembled with enough contiguity and annotated with enough accuracy, since a missing gene or incorrect gene boundaries, for example, can introduce errors into the comparisons.

An application example is the study of viral restriction factors such as OAS, PKR, APOBEC, TRIM or Mx. These genes, which help animal cells resist viral infection, are a useful model for studying how genes evolve under pressure. To study how these genes changed across chordates, from fish to mammals, a handful of genomes must be compared side by side, which is reliable only if those genomes are of sufficient assembly quality. Likewise, whether regulatory motifs in UTRs are conserved between species can only be studied if these UTRs are well annotated. Hence the need for well-assembled and well-annotated genomes.

These genomes are stored in databases, the major ones being NCBI, Ensembl, and NGDC/GWH. Each of them has its own formats, naming conventions and quality standards (see sections A to C). On top of that, the metadata describing the available sequencing datasets is often free text, inconsistent and hard to search systematically. As a result, selecting the right genomes and matching data by hand is slow, and nearly impossible to reproduce.

1.2 Objectives

The goal of this project is to build a web application that can be deployed locally. The tool collects, filters, and selects reference genomes, along with their associated raw sequencing data, across the three databases. This allows a reproducible, criteria-based search rather than a manual selection.

A search with this tool results only in lightweight statistics and metadata files. The genome sequences, annotations and reads are never downloaded: for every genome kept, the URLs of the associated artifacts are recorded instead, so that they remain retrievable if needed.

As a start, the development of the application is restricted to a single taxonomic group, Metazoa. Once an acceptable result is reached, an extension can be considered.

Beyond selecting reference genomes, the tool also aims to help identify representative raw sequencing data for the retained species: RNA-seq to begin with, though other data types could be added later.

In the longer term, the tool is meant to serve as a reusable foundation for comparative genomics projects that need a fast, trustworthy way to find and justify the genomes they work with.

1.3 Description of the existing

To our knowledge, there is currently no existing tool that performs this kind of genome selection automatically. What already exists are the three databases NCBI, Ensembl and NGDC/GWH, each accessible through its own API, with its own data structure and conventions.

Today, selecting genomes across these sources is done by hand: browsing each database separately and comparing statistics one by one, which is slow and hard to reproduce consistently.

Several challenges stem from the databases themselves. Ensembl's API has recently changed, including how its data is named and organized. GWH follows a different logic altogether, since it accepts incomplete or fragmented genome submissions, unlike the more structured releases of NCBI and Ensembl. NCBI itself splits its data into two parts: GenBank, which contains everything submitted, and RefSeq, a curated subset of higher-quality, reviewed assemblies. This project relies on RefSeq as a first quality filter.

As for the application itself, there is no existing software to build on: it will be developed as a new, locally deployable web tool.

1.4 Validation procedure and acceptance criteria

The following procedure describes the steps that will be followed to test the web application. Each step is validated when its acceptance criterion is met.

Step	Validation procedure	Acceptance criterion
1	The user enters the name of a species for which they want to find all annotated genomes.	The search accepts a common name, a scientific name or a taxonomic ID, for any taxonomic rank.
2	The user visualizes the results of the search as a table (entries) and as graphs (statistics).	The results can be filtered by assembly level and by thresholds (N50, BUSCO, number of genes, percentage of genes with annotated UTRs). Outliers and clusters are visualized with z-scores, MAD and PCA.
3	The user exports the raw data links for the selected genomes.	The export is in JSON format and can be easily used in external pipelines.
4	The user imports new statistics for each genome, computed with an external pipeline.	The imported file is in JSON format and is compatible with the internal data structure of the application.
5	Steps 2 to 5 are repeated with the new metrics.	The filtering, visualization and export work in the same way with the imported metrics as with the initial ones.

Table 1: Validation procedure and acceptance criteria.

2 Project requirements

2.1 Functional needs

The GenomeXplore web application allows the user, a bioinformatician, to explore the genome assemblies available in the three main databases (NCBI, Ensembl and GWH) through a taxonomic search. It comprises two complementary components: a backend dedicated to data collection and processing, and a frontend providing the graphical user interface. The overall architecture is detailed in section 3.

2.1.1 Backend – data collection and processing

The backend queries the APIs of NCBI, Ensembl and GWH to identify the assemblies associated with the selected taxon. For each assembly, it retrieves metadata, assembly and annotation statistics, and the URLs pointing to the available genomic and annotation files. A key challenge is to accommodate the differences between the APIs, in particular the query parameters, the response formats and the available information. The collected metadata and URLs are normalised into a common data structure and made available for the JSON report.

Additionally, the backend supports genome filtering and selection based on the available quality metrics, including N50, BUSCO, number of genes, and percentage of genes with annotated UTRs. These statistical parameters are processed through a PCA to visualise the clustering of assemblies, and are used to compute z-scores and MAD, with a cutoff designating outliers. Based on these calculations and visualisations, assemblies can then be filtered and used for the last step: retrieving the associated raw data.

For the selected assemblies, the backend queries SRA, ENA and GSA to identify experimental datasets such as RNA-seq, extending to ChIP-seq or Hi-C. Accession numbers and metadata are retrieved for each raw dataset, and a selection is proposed for each assembly before export.

2.1.2 Frontend – user interface

The frontend provides three pages (see Figure 4) corresponding to the successive stages of the workflow.

The **first page** is dedicated to entering a taxonomic query. A challenge here is to manage the format of the query (common or scientific name). The initial version of the application accepts scientific names; common-name handling may be added afterwards through a correspondence table.

The **second page** displays the retrieved assemblies, their metadata, their quality statistics and the URLs to the associated artifacts (FTP). Users can apply filters based on the statistics and select their preferred assemblies. The results of the PCA and of the z-score/MAD computation are displayed to visualise assembly clustering and outliers, helping the user decide whether to include specific assemblies. The selected assemblies, together with their metadata and artifacts, can be exported in JSON format.

The **third page** displays, for each selected assembly, the raw data available for the selected species. Users can choose the data type to export and inspect the run metadata. Selected runs and accessions are included in the final JSON export along with the assemblies they correspond to.

2.1.3 Points requiring further attention

The team will need to define the quality thresholds used for filtering, the metrics included in the statistical analyses, and how to treat assemblies with missing statistical values or artifacts.

Moreover, the project proposal stipulates that it would be interesting to implement normalised statistics precalculated outside the application; this could be explored for an alternate version.

The selection of experimental data, and more specifically the characterisation of the diversity of the data, relies primarily on text embeddings, which remain optional for now.

2.2 Non-functional needs

The GenomeXplore web application must be deployable locally and offer a simple, documented installation procedure, based on dependency management using a single container. Exchanges with external APIs must comply with the request limits imposed by each source. JSON exports must be versioned and self-descriptive to facilitate their interpretation and use in external workflows, such as the calculation of normalised statistics suggested in the project. The application must handle communication errors and the unavailability of external services (databases), and inform the user of the nature of the difficulty encountered.

3 Solution architecture

This section presents the architecture of GenomeXplore at three levels, following the C4 model: the whole system (section 3.1), the backend (section 3.2) and the frontend (section 3.3). The user interface itself is described in section 3.4.

3.1 Overall architecture

Figure 1 shows GenomeXplore at the highest level. The whole system runs locally on the researcher's machine and is made of two containers (in the C4 sense of an independently runnable unit): a JavaScript **frontend** and a Python **backend API** built with FastAPI. They communicate through JSON over HTTPS.

The researcher interacts only with the frontend: they enter a search (scientific name, genome ID, ...), explore and select genomes, import custom statistics computed from their own analyses, and export the raw annotation links of the selected genomes. The backend is the only component talking to the outside world: it sends read-only REST GET requests to NCBI, Ensembl and GWH, combines their answers, applies simple pre-filters, computes the statistics and returns the annotation links.

In accordance with the project constraints, only metadata, statistics and URLs transit through the system; no sequence, annotation or read file is downloaded.

3.2 Backend architecture

The backend (Figure 2) is organised as a processing chain. Each component has a single responsibility:

- **API endpoints** (FastAPI routes): receive the requests from the frontend and return the results as JSON. They forward search requests to the database queries and raw-data requests to the raw data links component.
- **Database queries**: one specific query module per database (NCBI, Ensembl, GWH), each adapted to the API, parameters and response format of its source.
- **Aggregation and pre-filters**: combine the three responses into a common structure and apply simple pre-filters (for instance on the assembly level).
- **Statistics**: compute the data required by the charts, e.g. the PCA and the BUSCO-related values.
- **Raw data links**: extract the raw annotation links from the responses of the databases, to be returned to the frontend and exported.

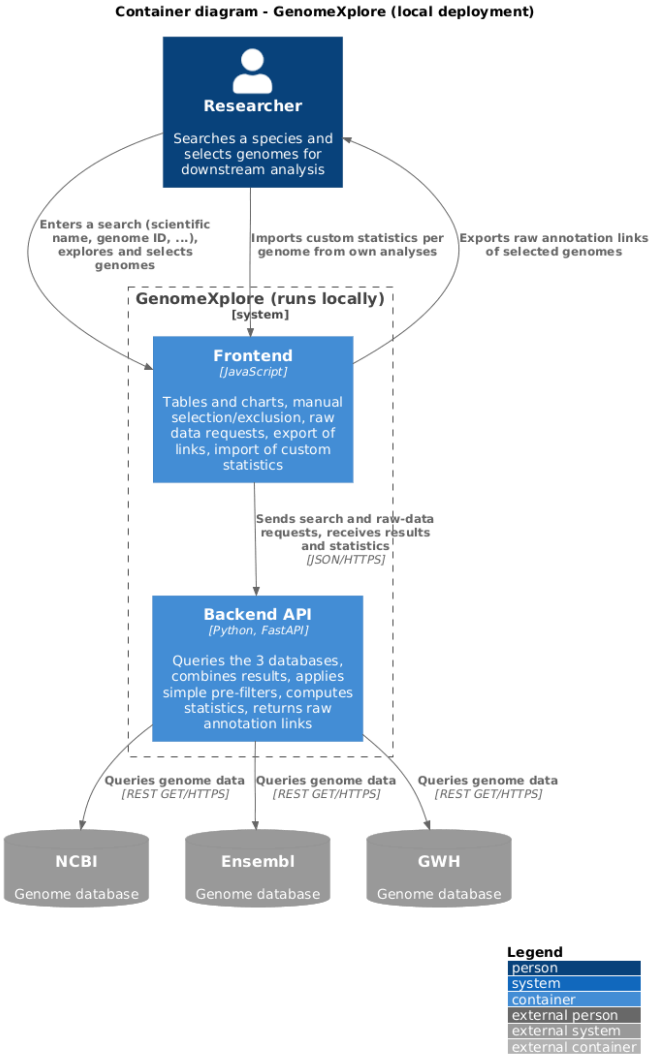


Figure 1: Container diagram of GenomeXplore (local deployment).

Isolating one query module per database makes it possible to handle the differences between the sources (see section 2.1.1) and to add or replace a source without modifying the rest of the chain. The characteristics of each source are documented in sections A to C.

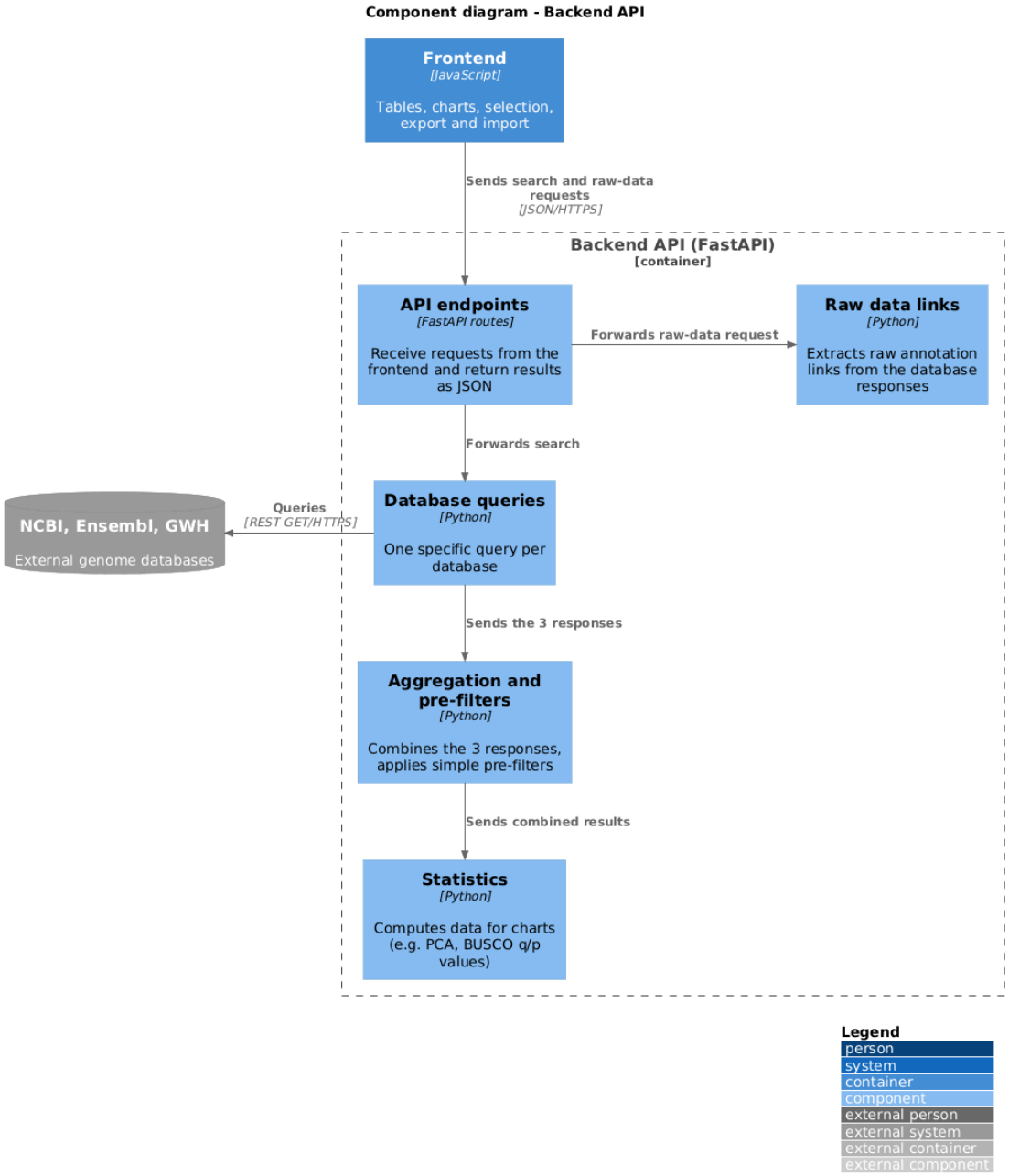


Figure 2: Component diagram of the backend API.

3.3 Frontend architecture

The frontend (Figure 3) is made of three components, which correspond to the main actions of the researcher:

- **Search**: the search form (scientific name, genome ID, ...) which sends the search requests to the backend API.

- **Tables and charts**: display the results and the statistics, and allow the manual selection or exclusion of genomes. This component also sends the raw-data requests to the backend and receives the results and statistics.
- **Import / export**: imports custom statistics computed outside the application, and exports the raw annotation links of the selected genomes. It exchanges the selected genomes and the imported statistics with the tables and charts component.

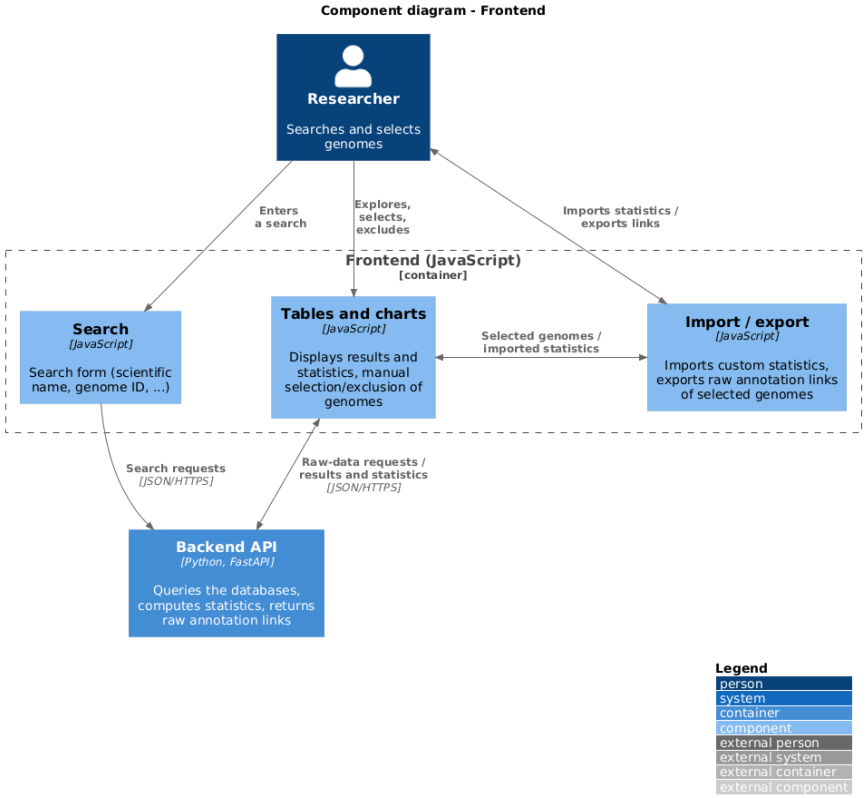


Figure 3: Component diagram of the frontend.

3.4 User interface design

The user interface is organised in three pages, following the workflow of the application. The mock-up is given on the next page (Figure 4):

1. **Page 1 – Taxonomic search**: entry of a scientific name, with a short description of the application.
2. **Page 2 – Assemblies exploration and selection**: table of the assemblies from the three sources with their statistics, interactive PCA and z-score/MAD plots to spot outliers, selection for export, and a first JSON export containing the assembly URLs only.
3. **Page 3 – Raw data exploration and selection**: raw data (RNA-seq, ChIP-seq, Hi-C, ...) available for the selected assemblies with their run metadata, and the final JSON export combining assemblies and corresponding raw data. A space is reserved for the results of the embeddings in a later version.

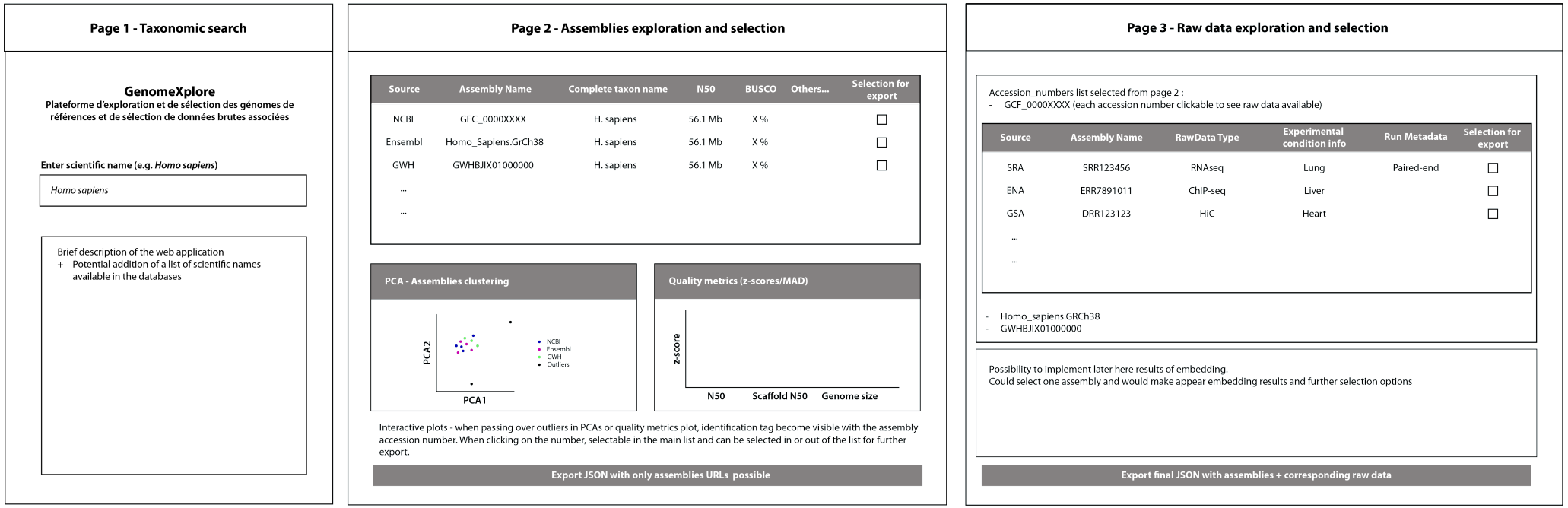


Figure 4: Web page design (mock-up): taxonomic search (page 1), assemblies exploration and selection (page 2), raw data exploration and selection (page 3).

Page 1 Scientific-name input field and description of the application.

Page 2 Assemblies table (source, assembly name, taxon, N50, BUSCO, ...), PCA clustering, quality metrics (z-scores/MAD), selection checkboxes and JSON export of the assembly URLs.

Page 3 Raw data table (source, run accession, data type, experimental condition, run metadata), selection checkboxes and final JSON export.

4 Constraints

4.1 Budget and resources

No budget is allocated to this project, and none is required at this stage. The project therefore relies exclusively on open-source software, libraries and tools, and on freely accessible APIs and databases. No licensed or paid solutions can be used.

Since the application is designed for local deployment only, there are no specific hardware requirements. If hosting on a server or other infrastructure becomes necessary during or after the project, a budget and material resources would have to be defined at that point.

4.2 Schedule

The project is divided into three main stages, each with its own time frame:

- Specifications book: 2 weeks;
- Tool development: 2 weeks;
- Finalization and defense: 2 weeks.

The project has a fixed number of dedicated weeks. During these weeks, the team still has a few courses to attend, which slightly reduces the time available for the project. Outside of these weeks, regular classes take place and the team cannot work on the project. The team organizes its time accordingly to make sure that all deliverables are completed on schedule. The detailed planning is given in Figure 5 (section 5.1).

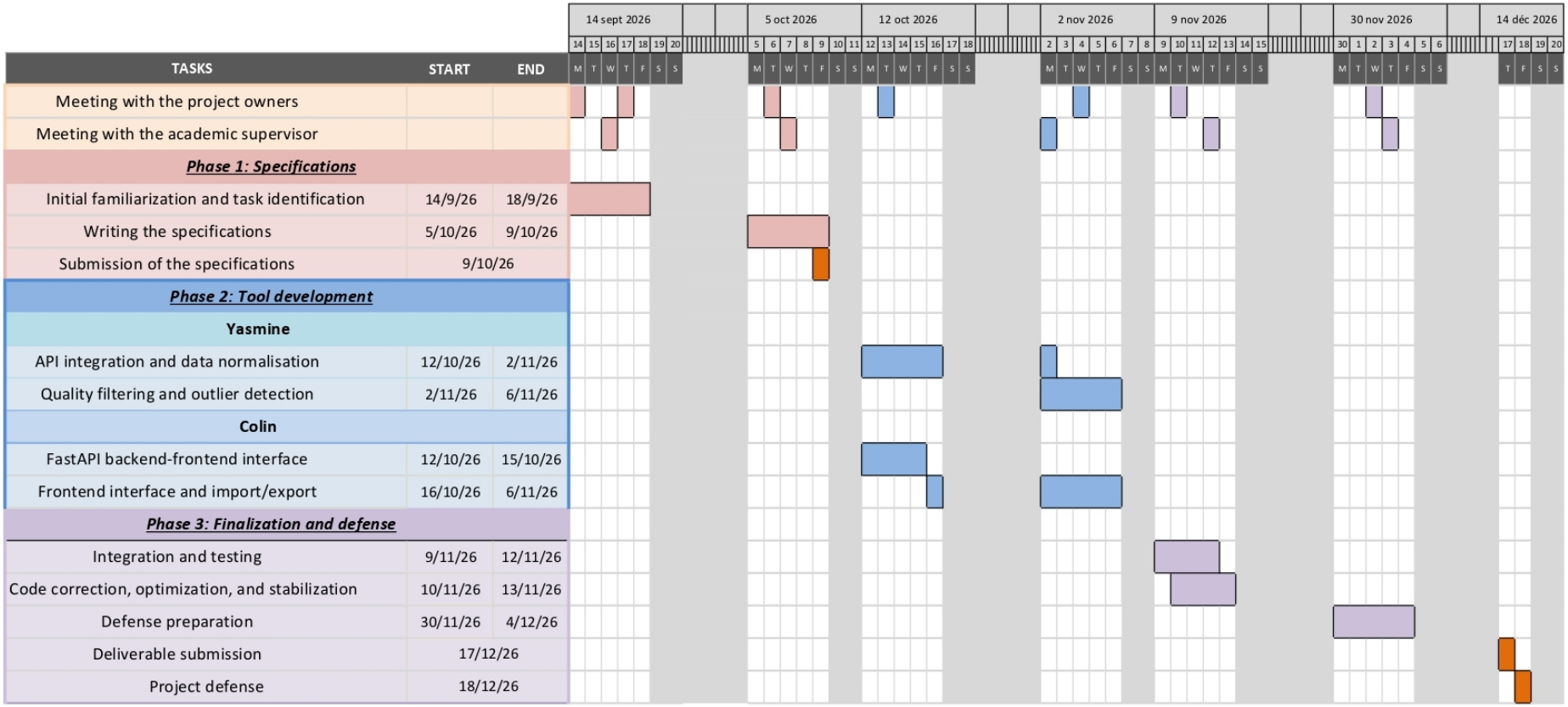
4.3 Other constraints

The application depends on external APIs and databases, so the team is limited to the data and information they make accessible. The number of requests allowed and their timing (rate limits, availability) also constrain how the data can be collected and processed.

5 Project development process

5.1 Planning

The planning of the project is presented as a Gantt chart on the next page (Figure 5).



TASKS	START	END
Meeting with the project owners		
Meeting with the academic supervisor		
Phase 1: Specifications		
Initial familiarization and task identification	14/9/26	18/9/26
Writing the specifications	5/10/26	9/10/26
Submission of the specifications	9/10/26	
Phase 2: Tool development		
Yasmine		
API integration and data normalisation	12/10/26	2/11/26
Quality filtering and outlier detection	2/11/26	6/11/26
Colin		
FastAPI backend-frontend interface	12/10/26	15/10/26
Frontend interface and import/export	16/10/26	6/11/26
Phase 3: Finalization and defense		
Integration and testing	9/11/26	12/11/26
Code correction, optimization, and stabilization	10/11/26	13/11/26
Defense preparation	30/11/26	4/12/26
Deliverable submission	17/12/26	
Project defense	18/12/26	

Figure 5: Gantt chart of the project (specifications, tool development, finalization and defense).

5.2 Quality control and testing procedures

The following procedures are adopted to control the quality of the software:

- **Benchmarking**: the application is benchmarked on concrete use cases to assess its behavior and performance across its different functionalities;
- **Robustness testing**: we deliberately try to break the application, for example with invalid inputs, unexpected usage or unavailable data sources, to check that it handles errors correctly and is not prone to bugs;
- **Scenario testing**: all the usage scenarios of the application are tested to make sure each functionality works as intended;
- **Validation on real cases**: concrete examples provided by the bioinformatics and biostatistics teams are used to verify that the application produces correct results in real conditions.

5.3 Documentation

Alongside the application, the shared GitLab repository contains the following documentation, intended for the bioinformatics and biostatistics teams:

- a README file providing an overview of the application;
- deployment instructions explaining how to install and run the application locally;
- annex files providing a detailed description of the databases and APIs used (sections A to C);
- benchmarking and test examples, showing the application in use across its different functionalities with concrete examples.

6 Responsibilities

6.1 Project owner

This project is managed by the Bioinformatique et Biostatistique team of the Centre International de Recherche en Infectiologie (CNRS UMR5308 – INSERM U1111 – UCBL1 – ENSL), under the supervision of Julien FOURET and Carine REY. As project owner, they are responsible for:

- expressing the need and defining the objectives of the project;
- monitoring the progress of the project and assessing whether the pace of work is adequate, given the limited time the students can dedicate to it;
- reviewing the deliverables and giving feedback and suggestions for improvement;
- sharing their expertise on specific topics to support the team;
- validating the final product by confirming that it meets their expectations and needs.

6.2 Project delivery team

This project is carried out by M2 Bioinformatics students from Lyon 1 University (Yasmine ALIOUAT, Colin PALANDRE and Margaux PILLON) as part of the "Projet Bioinformatique 3" course unit. This book of specifications was written by all three of us; the remainder of the project will be carried out by only two of us (Yasmine ALIOUAT and Colin PALANDRE). As the project delivery team, we are responsible for:

- designing the solution, including the technical choices and the software architecture;
- developing the web application and implementing its functionalities;
- testing the application;
- writing the documentation associated with the project;
- delivering the project in line with the deadlines, quality requirements and objectives defined by the project owner.

A NCBI database description

Source sheet describing the internal organisation, pipelines, versioning and access points of NCBI (RefSeq/GenBank). The following pages reproduce the document.

NCBI Documentation Source Sheet

1. Role in GenomeXplore

NCBI (United States) provides genome assemblies, annotations and metadata. We use two access points, each with a different role:

Access point	Role
Datasets REST API https://api.ncbi.nlm.nih.gov/datasets/v2/	Find genomes by taxon (name, common name or taxid); get assembly-level metadata
FTP https://ftp.ncbi.nlm.nih.gov/genomes/	Location of the heavy files (FASTA, GFF/GTF, ...) and of the detailed statistics files (N50, etc.)

2. Internal organisation

NCBI stores genome assemblies in two parallel collections:

- **GenBank**: raw, unfiltered submissions from any group. Accessions start with `GCA_`.
- **RefSeq**: a curated subset of GenBank, re-processed and selected by NCBI staff. Accessions start with `GCF_`.

A given assembly often exists under both forms (a raw GenBank version and its curated RefSeq counterpart), linked to each other.

3. API access

Request: `GET /datasets/v2/genome/taxon/{taxon}/dataset_report`

- `taxon`: accepts a taxonomy id (e.g. `9685`), a scientific name (e.g. `felis catus`), or a common name (e.g. `human`), at any taxonomic rank (species, genus, family, ...).
- Returns one entry per assembly found under that taxon (so a higher-rank taxon such as a family returns every assembly for every species underneath it).
- Rate limit: 5 requests/second by default, 10 requests/second with a (free) API key.

Example

URL: https://api.ncbi.nlm.nih.gov/datasets/v2/genome/taxon/felidae/dataset_report

Response (one entry of the `reports` list, shortened):


```
{
  "accession": "GCA_018350175.1",
  "paired_accession": "GCF_018350175.1",
```

```
  "source_database": "SOURCE_DATABASE_GENBANK",
  "organism": {
    "tax_id": 9685,
    "organism_name": "Felis catus",
    "common_name": "domestic cat"
  },
  "assembly_info": {
    "assembly_level": "Chromosome",
    "assembly_name": "F.catus_Fca126_mat1.0",
    "release_date": "2021-05-13",
    "submitter": "Texas A&M University",
    "refseq_category": "reference genome",
    "sequencing_tech": "PacBio Sequel 2",
    "assembly_method": "NextDenovo v. 2.2-beta.0"
  },
  "bioproject_accession": "PRJNA684600"
}
```


What this endpoint does and does not give. It gives assembly-level identity and provenance metadata (accession, paired GenBank/RefSeq accession, organism, assembly level, sequencing technology, submitter, dates, BioProject/BioSample links). It does **not** include the N50 or the direct URLs to the data files (FASTA, GFF, ...): those are only found on the FTP side, inside the per-assembly folder (see below). In practice the API is best suited to the first step (searching a taxon and listing its assemblies); the FTP side is then used to fetch the statistics and artifact URLs for the assemblies that are kept.

FTP has no API. Unlike the Datasets endpoint above, the FTP side (https://ftp.ncbi.nlm.nih.gov/genomes/) is a plain file/directory listing served over HTTPS, not a queryable API: there is no search parameter, only a fixed path to browse or fetch directly (see the next sections). No login or API key is required to browse or download these files.

4. Finding assemblies and associated data files

Assemblies are organised by broad taxonomic group, then by species, then by individual assembly.

```
/genomes/refseq/   (or /genomes/genbank/ for the raw collection)
    archaea/
    bacteria/
    fungi/
    invertebrate/
    plant/
    protozoa/
    vertebrate_mammalian/
    vertebrate_other/
    viral/
```

Within each group, one folder per species:

```
vertebrate_mammalian/
    <Genus_species>/
        assembly_summary.txt
        annotation_releases/
        <Accession.version>_<assembly_name>/
```

Example path (*Felis catus*, domestic cat):

```
/genomes/refseq/vertebrate_mammalian/Felis_catus/
    GCF_018350175.1_F.catus_Fca126_mat1.0/
```

The accession is also split into groups of 3 digits to build the full path under `/genomes/all/`, e.g.:

```
https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/018/350/175/
GCF_018350175.1_F.catus_Fca126_mat1.0/
```

This path does not need to be built by hand: it is given directly in the `ftp_path` column of `assembly_summary.txt` (see below).

Inside each assembly folder:

```
<Accession.version>_<assembly_name>_genomic.fna.gz      genome FASTA
<Accession.version>_<assembly_name>_genomic.gff.gz      annotation (GFF3)
<Accession.version>_<assembly_name>_genomic.gtf.gz      annotation (GTF)
<Accession.version>_<assembly_name>_protein.faa.gz      proteins
<Accession.version>_<assembly_name>_rna.fna.gz          RNA / transcripts
<Accession.version>_<assembly_name>_cds_from_genomic.fna.gz   coding sequences
<Accession.version>_<assembly_name>_assembly_report.txt
<Accession.version>_<assembly_name>_assembly_stats.txt
<Accession.version>_<assembly_name>_feature_count.txt.gz
md5checksums.txt
```

5. Finding statistics files

`assembly_summary.txt` (one per species folder) lists every assembly for that species in a single table, with the main metadata already filled in: accession, taxid, assembly level, submitter, release date, `ftp_path`, genome size, GC%, scaffold/contig count, and gene counts.

It does **not** include N50. N50 (contig-N50, scaffold-N50) is only found inside the per-assembly file `assembly_stats.txt`, together with other details such as sequencing technology and assembly method.

Example, *Felis catus* (GCF_018350175.1):

- Assembly level: Chromosome
- Genome size: 2,425,730,029 bp
- GC content: 42.0%
- Total gene count: 39,307
- Protein-coding gene count: 20,453
- Contig N50: (from `assembly_stats.txt`)
- Sequencing technology: PacBio Sequel 2

6. Differences between NCBI and GWH

NCBI assembly folders are generally richer than GWH direct submissions: NCBI typically provides GFF and GTF (both formats), an assembly report, a dedicated statistics file with N50, a feature count table and an XML annotation report, in addition to genome/RNA/CDS/protein FASTA files.

GWH direct submissions usually provide a simpler fixed set of files (genome FASTA, GFF, RNA, CDS, proteins, features, MD5) and statistics are shown mainly on the web page rather than in

a dedicated stats file.

7. Internal pipelines

Assembly release (RefSeq)

1. Raw sequencing data and the resulting assembly are submitted to NCBI (GenBank).
2. NCBI evaluates the assembly; a subset meeting NCBI's criteria is selected and curated for RefSeq.
3. A RefSeq accession (`GCF_`) is assigned and linked to the original GenBank accession (`GCA_`).
4. The assembly is annotated (see below) and the files are released under `/genomes/refseq/`.

Annotation pipeline

1. Gene prediction and annotation are run on a given assembly (NCBI's eukaryotic annotation pipeline, or RefSeq prokaryotic pipeline).
2. The result is published as a numbered "Annotation Release" folder.
3. A later annotation release can be produced on the same assembly (new evidence, pipeline update) or on a newer assembly version.

8. Updates to assemblies and annotations

Assembly versions are indicated by the number after the dot in the accession:

```
accession_core = GCF_018350175
version        = 1
accession      = GCF_018350175.1
```

A new assembly submission for the same effort increases this version number.

Annotation releases are numbered independently of the assembly version, in sequential order, starting from an arbitrary number and increasing by one each time a new release is produced:

```
Felis_catus/annotation_releases/
    104/       (older release)
    105/       (current release)
    current/   (always points to the latest release)
```

The full accession, including the version suffix, should always be kept when referencing an assembly, since different versions of the same accession can correspond to different (though related) assemblies.

B Ensembl database description

Source sheet describing the internal organisation, pipelines, versioning and access points of Ensembl. The following pages reproduce the document.

Source sheet: Ensembl (GenomeXplore)

1. Role in GenomeXplore

Ensembl (EMBL-EBI, Europe) provides genome assemblies, annotations and metadata. We use three access points, each with a different role:

Access point	Role
REST search `https://www.ensembl.org/api/search/genomes/v3`	Find genomes from any of 5 key types; get summary stats and the FTP URL
GraphQL `https://www.ensembl.org/data/graphql`	Detailed metadata per genome (assembly, organism, datasets)
FTP `http://ftp.ebi.ac.uk/pub/ensemblorganisms/...`	Location of the heavy files (FASTA, GTF/GFF, ...)

2. REST search endpoint

Request: `GET /api/search/genomes/v3?query=<text>&page=<n>&per_page=<n>`

- `query`: free text. Accepts common name, scientific name, taxonomy id, assembly accession / genome tag, and genome id (UUID).
- `page`, `per_page`: pagination.

Example

URL: `https://www.ensembl.org/api/search/genomes/v3?query=Homo sapiens&page=1&per_page=1`

Response:


```
{
  "meta": {"total_hits": 563, "page": 1, "per_page": 1},
  "matches": [{
    "genome_id": "59871324-7803-4234-856e-2a2bd96d7b3c",
    "genome_tag": "GCA_000001405.29",
    "species_taxonomy_id": "9606",
    "taxonomy_id": "9606",
    "common_name": "Human",
    "scientific_name": "Homo sapiens",
    "type": null,
    "assembly": {
      "accession_id": "GCA_000001405.29",
      "name": "GRCh38.p14",
      "url": "https://identifiers.org/insdc.gca/GCA_000001405.29"
```

```
    },
    "is_reference": true,
    "coding_genes_count": 20543,
    "contig_n50": 54806562,
    "has_variation": true,
    "has_regulation": true,
    "annotation_provider": "GENCODE",
    "organism_id": "1d336185-affe-4a91-85bb-04ebd73cbb56",
    "annotation_method": "Manual annotation",
    "rank": 1,
    "release": {"name": "2026-07", "type": "integrated"},
    "ftp_url":
"http://ftp.ebi.ac.uk/pub/ensemblorganisms/GCA/000/001/405/29/ensembl/
2026_04/"
  }]
}
```

3. GraphQL endpoint

Root queries: `version`, `genome`, `genomes`, plus gene-level queries (`gene`, `genes`, `transcript`, `transcript_search`, `protein_search`, `product`, `overlap_region`, `region`). Only `genome` and `genomes` are needed to find genomes; the others work inside a genome.

3.1 Looking up genomes

Query	Argument	Returns
`genome`	`by_genome_id: {genome_id: "<uuid>"}`	one genome
`genomes`	`by_keyword: {<key>: "<value>"}`	a list of genomes

Keys of `by_keyword`: `tolid`, `assembly_accession_id`, `assembly_name`, `ensembl_name`, `common_name`, `scientific_name`, `scientific_parlance_name`, `species_taxonomy_id` (String), `release_version` (Float).

Rules:

- Exactly one key per query.
- Exact match only (no partial matching); the assembly name includes the patch level, e.g. `GRCh38.p14`.
- The genome tag cannot be searched in GraphQL.

Example

Query:

```
query {
  genome(by_genome_id: {genome_id: "59871324-7803-4234-856e-
2a2bd96d7b3c"}) {
```

```
    genome_id assembly_accession scientific_name taxon_id genome_tag
is_reference
  }
}
```

Response:


```
{
  "data": {
    "genome": {
      "genome_id": "59871324-7803-4234-856e-2a2bd96d7b3c",
      "assembly_accession": "GCA_000001405.29",
      "scientific_name": "Homo sapiens",
      "taxon_id": 9606,
      "genome_tag": "GRCh38",
      "is_reference": true
    }
  },
  "extensions": {
    "execution_time_in_seconds": 1.14
  }
}
```


3.2 Fields returned (type Genome)

`genome_id`, `assembly_accession`, `scientific_name`, `release_number` (Float), `release_date`, `taxon_id`, `parlance_name`, `genome_tag`, `is_reference`, `assembly { assembly_id name accession_id accessioning_body default tolid organism { id scientific_name scientific_parlance_name tol_id is_reference_organism species { taxon_id scientific_name ncbi_common_name alternative_names } } }`, `dataset { dataset_id name type dataset_type source version release release_date release_type }`.

3.3 Which route for which search

User input	Route
common name	GraphQL `common_name` or REST
scientific name	GraphQL `scientific_name` or REST
taxonomy id	GraphQL `species_taxonomy_id` or REST
assembly accession	GraphQL `assembly_accession_id` or REST
genome id	GraphQL `genome(by_genome_id)` or REST
genome tag	REST only

REST search handles all five key types in one call, so it is the proposed single entry point; GraphQL is then called by `genome_id` for the detailed metadata.

4. FTP (heavy files)

The FTP has no API: it is a plain directory listing served over HTTP/FTP. Example for the human reference:

`http://ftp.ebi.ac.uk/pub/ensemblorganisms/GCA/000/001/405/29/ensembl/2026_04/`

The path is built from the assembly accession (`GCA_000001405.29` becomes `GCA/000/001/405/29`), followed by `ensembl` and the release folder. There is no need to rebuild it: the REST search already returns the full URL in the `ftp_url` field, and that is the value stored in the JSON export.

All the files available in a folder can be detected by listing the directory, but this is slow, so it is not ideal for the collector. We only give the `ftp_url`, and users download the raw files they need from there.

C NGDC/GWH database description

Source sheet describing the internal organisation, pipelines, versioning and access points of NGDC/GWH (Genome Warehouse). The following pages reproduce the document.

GWH – Source Documentation Sheet

GWH — Source Documentation Sheet

1. Internal organisation

Genome Warehouse (GWH) hosts genome assemblies submitted directly to GWH, as well as assemblies integrated from **NCBI GenBank and RefSeq**.

Downloadable files are hosted on the CNCB download server. Their directory structure depends on their origin:

Data source	Download directory
Assemblies submitted directly to GWH	`/gwh/`
Reannotations produced by GWH	`/gwh/Reannotation/`
Assemblies integrated from NCBI	`/assembly/ncbi/`

1.1 Finding assemblies and associated files

Assemblies submitted directly to GWH Each assembly is associated with a **BioProject** and a **BioSample**:

Record	Role	Accession prefix
BioProject	Describes the research project	`PRJCA...`
BioSample	Describes the biological sample	`SAMC...`
Assembly	Identifies the submitted genome assembly	`GWH...`

A BioProject may include several BioSamples, and a BioSample may be associated with several assemblies.

Files are available under:

```
https://download.cncb.ac.cn/gwh/
```

Assemblies are organised into broad organism categories: `Animals`, `Archaea`, `Bacteria`, `Fungi`, `Metagenomes`, `Others`, `Plants`, `Protists` and `Viruses`.

The typical path to an assembly directory is:

```
/gwh/<Organism_group>/<Scientific_name>_<Assembly_name>_<GWH_accession>/
```

Depending on whether annotations have been provided, the directory may contain the following files:

Filename pattern	Content
<GWH_accession>.genome.fasta.gz	Genome assembly sequences
<GWH_accession>.gff.gz	Genome annotation
<GWH_accession>.RNA.fasta.gz	RNA sequences
<GWH_accession>.CDS.fasta.gz	Coding sequences
<GWH_accession>.Protein.faa.gz	Protein sequences
<GWH_accession>.feature.gz	Feature file
<GWH_accession>_md5.txt	MD5 checksums

File availability varies between assemblies. An assembly without annotation will not necessarily include RNA, CDS or protein files.

Reannotations produced by GWH GWH reannotations are stored separately:

```
https://download.cncb.ac.cn/gwh/Reannotation/
```

The observed directory structure is:

```
/gwh/Reannotation/<Organism_group>/<Two-letter_subdirectory>/<Reannotation_accession>/
```

Example:

```
/gwh/Reannotation/Bacteria/DU/GWHRA_DUBW00000000.2/
```

The directory may contain genome sequences, annotations, RNA, CDS, proteins, feature files and MD5 checksums. Filenames use the reannotation accession as their prefix.

For example:

```
GWHRA_DUBW00000000.2.genome.fasta.gz
GWHRA_DUBW00000000.2.gff.gz
GWHRA_DUBW00000000.2.RNA.fasta.gz
GWHRA_DUBW00000000.2.CDS.fasta.gz
GWHRA_DUBW00000000.2.Protein.faa.gz
GWHRA_DUBW00000000.2.feature.gz
```

Assemblies integrated from NCBI Assemblies integrated from NCBI are available under:

```
https://download.cncb.ac.cn/assembly/ncbi/
```

They are organised first by organism group, then by their source accession prefix:

Prefix	Original database
GCA_	NCBI GenBank
GCF_	NCBI RefSeq

Within these directories, numerical subdirectories are derived from the accession number.

The observed path structure is:

```
/assembly/ncbi/<Organism_group>/<GCA_or_GCF>/<First_three_digits>/<Next_three_digits>/<Assembly_directo
```

Example:

```
/assembly/ncbi/Animals/GCA/000/002/Apis_mellifera_honey_bee_Amel_4.5_GCA_000002195.1/
```

The final directory contains the files available for that assembly. Observed examples include:

```
<GCA_or_GCF_accession.version>_<Assembly_name>_genomic.fna.gz
<GCA_or_GCF_accession.version>_md5.txt
```

Additional files may be present. Their availability should be checked for each assembly rather than assumed to match the complete original NCBI directory.

Identifying record types from accession prefixes The following prefixes help distinguish the origin and type of records:

Accession prefix	Meaning
`GWH...`	Assembly submitted directly to GWH
`GWHRA_...`	Reannotation produced by GWH
`GCA_...`	GenBank assembly integrated from NCBI
`GCF_...`	RefSeq assembly integrated from NCBI

1.2 Finding statistics and metadata

Assembly statistics are displayed on GWH assembly pages. Their availability and level of detail vary between records.

Dedicated statistics files were not identified in the assembly download directories inspected. However, this observation should not be interpreted as evidence that all statistics are available exclusively through the website.

GWH also provides a public JSON API:

```
https://ngdc.cncb.ac.cn/gwh/api/public/assembly/<Assembly_accession>
```

The documented response includes assembly metadata and links to available files. The availability of individual quality metrics should be checked separately.

Example: `GWHBJIX01000000` — *Canis lupus familiaris*

The assembly page displays:

Statistic	Value
Genome size	16,751 bp
GC content	39.68%

Although the record is labelled “Complete genome” and “Full Genome”, this sequence length should not be interpreted as the size of the complete nuclear genome of the species. The type and scope of the deposited sequence must be checked before including it in comparative analyses.

1.3 Differences between NCBI and GWH download directories

NCBI assembly directories may provide a wider range of reports and annotation-related files than the GWH directories inspected.

File or information type	NCBI assembly directories	Direct GWH assembly directories
Genome FASTA	Available	Available
GFF annotation	When available	When available
GTF annotation	When available	Not identified in the inspected examples
RNA, CDS and protein sequences	When available	When available
Assembly reports and statistics files	Often provided	Not identified in the inspected examples
Annotation reports and supporting evidence	Depending on the annotation	Availability requires verification
Feature files	When available	When available
Checksums	Provided	Provided

For GWH, assembly pages and API responses complement the downloadable files. Quality control also forms part of the submission process.

2. Internal pipelines

2.1 Submission pipeline

The submitter performs the genome assembly before submitting it to GWH.

The main submission stages are:

1. **Assembly preparation**
   The submitter assembles sequencing reads obtained from GSA or another source.
2. **File and metadata preparation**
   The submission includes the assembly sequences in FASTA format and the required metadata. Annotation files may also be supplied.
3. **Association with project and sample records**
   The assembly is linked to a BioProject (`PRJCA...`) and a BioSample (`SAMC...`).
4. **Submission and quality control**
   GWH performs automated checks. The submitter may need to correct files or metadata before the submission is accepted.
5. **Accession assignment and release**
   The accepted assembly receives a GWH accession and is released according to the applicable release conditions.

GWH also provides an automated **prokaryotic genome reannotation pipeline based on NCBI PGAP**. Reannotation results receive a separate accession and are made available in the `Reannotation` download directory.

2.2 Updates to assemblies and annotations

For directly submitted assemblies, the initial accession may appear without a dot-version suffix:

```
GWHAAEV00000000
```

Updates are identified by an incremental suffix:

```
GWHAAEV00000000.1
GWHAAEV00000000.2
```

For data processing, the accession can therefore be separated into a stable core and an update version:

```
accession_core = GWHAAEV00000000
update_version = 2
full_accession = GWHAAEV00000000.2
```

The absence of a suffix must be preserved explicitly; it should not automatically be interpreted as `.1`.

Reannotation accessions may also include version suffixes, for example:

```
GWHRA_DUBW00000000.2
```

Assembly and reannotation identifiers should be stored separately, with their full versioned accessions, so that the original assembly and its associated annotation can be distinguished.